

```
#!/usr/bin/env python3
"""
Rede de sensores para alerta antecipado de
desastre – protótipo real
=====
Sem holon, sem telepatia, sem phi_S.
Apenas:
    - Deteccao de anomalia estatistica (z-
score sobre linha de base movel)
    - Fusao de dois canais reais: fisiologico
(HRV, proxy de estresse) e
    ambiental (temperatura)
    - Corroboracao espacial: exige
concordancia entre varios sensores antes
    de promover a um alerta de comunidade -
- essa e a mesma tecnica usada
    de verdade em redes sismicas e de
enchente para reduzir falso positivo
    - Logica de baixa banda: cada no so
transmite 1 bit (alerta/nao-alerta),
    nunca o sinal bruto -- e o que uma rede
LoRaWAN real faria
"""

import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

N_NODES = 20
```

```

T = 4320                # 3 dias completos
(1440 min/dia)
DIURNAL_PERIOD = 1440 # periodo real do
ciclo diurno (1 dia), independente de T
BASELINE_WINDOW = 60
Z_THRESHOLD = 2.5
CORROBORATION_MIN = 4
CALIBRATION_END = 2880 # 2 dias completos
de calibracao antes do evento

EVENT_START = 3200      # evento começa no
3o dia, depois da calibracao completa
EVENT_RAMP = 80

#
=====
=====
# SIMULACAO DOS DADOS BRUTOS POR NO (nunca
saem do dispositivo)
#
=====
=====
def simulate_node_data(node_id, seed):
    rng = np.random.RandomState(seed)
    t = np.arange(T)
    temp_baseline = 25 +
3*np.sin(2*np.pi*t/DIURNAL_PERIOD) +
rng.randn(T)*0.5
    hrv_baseline = 55 + rng.randn(T)*6

    temp = temp_baseline.copy()
    hrv = hrv_baseline.copy()

    # injeta o evento real (onda de calor):

```

```

temperatura sobe, HRV cai (estresse
fisiologico)
    # cada no tem uma sensibilidade/atraso
    levemente diferente (heterogeneidade real)
    delay = rng.randint(-15, 15)
    intensity = rng.uniform(0.7, 1.3)
    for i, tt in enumerate(t):
        if tt > EVENT_START + delay:
            progress = min(1.0, (tt -
EVENT_START - delay) / EVENT_RAMP)
            temp[i] += intensity * 6.0 *
progress          # +6 C no pico
            hrv[i] -= intensity * 20.0 *
progress          # -20ms no pico (mais
estresse)

    return temp, hrv

nodes_temp = []
nodes_hrv = []
for n in range(N_NODES):
    temp, hrv = simulate_node_data(n,
seed=100+n)
    nodes_temp.append(temp)
    nodes_hrv.append(hrv)
nodes_temp = np.array(nodes_temp) #
(N_NODES, T)
nodes_hrv = np.array(nodes_hrv)

#
=====
=====
# ALGORITMO DE BORDA (roda dentro de cada
'traje'/sensor, real)

```

```

#
=====
=====
def edge_zscore(series, t,
window=BASELINE_WINDOW):
    """Z-score da leitura atual vs linha de
base movel anterior (causal, sem 'ver o
futuro')."""
    lo = max(0, t - window)
    baseline = series[lo:t]
    if len(baseline) < 10:
        return 0.0
    mu, sd = baseline.mean(),
baseline.std()
    if sd < 1e-6:
        return 0.0
    return (series[t] - mu) / sd

node_alerts = np.zeros((N_NODES, T),
dtype=bool)
node_risk = np.zeros((N_NODES, T))

def fit_diurnal_model(series, calib_end):
    """Ajusta  $a+b\sin(2\pi t/\text{periodo})+c\cos(2\pi t/\text{periodo})$  usando a
fase de calibracao."""
    t_calib = np.arange(calib_end)
    X =
np.column_stack([np.ones(calib_end),
np.sin(2*np.pi*t_calib/DIURNAL_PERIOD),
np.cos(2*np.pi*t_calib/DIURNAL_PERIOD)])
    coef, _, _, _ = np.linalg.lstsq(X,
series[:calib_end], rcond=None)
    t_full = np.arange(T)

```

```

        X_full = np.column_stack([np.ones(T),
np.sin(2*np.pi*t_full/DIURNAL_PERIOD),
np.cos(2*np.pi*t_full/DIURNAL_PERIOD)])
        return X_full @ coef # modelo diurno
estimado para toda a serie

for n in range(N_NODES):
    temp_model =
fit_diurnal_model(nodes_temp[n],
CALIBRATION_END)
    hrv_model =
fit_diurnal_model(nodes_hrv[n],
CALIBRATION_END)
    temp_resid = nodes_temp[n] -
temp_model # residuo: o que sobra alem do
padrao diurno conhecido
    hrv_resid = nodes_hrv[n] - hrv_model
    # desvio-padrao do residuo, estimado so
na fase de calibracao (linha de base de
ruído)
    temp_resid_sd =
temp_resid[:CALIBRATION_END].std() + 1e-6
    hrv_resid_sd =
hrv_resid[:CALIBRATION_END].std() + 1e-6
    for t in range(T):
        z_temp = temp_resid[t] /
temp_resid_sd
        z_hrv = -hrv_resid[t] /
hrv_resid_sd
        risk = max(z_temp, 0) + max(z_hrv,
0)
        node_risk[n, t] = risk
        node_alerts[n, t] = risk >
Z_THRESHOLD

```

```

#
=====
=====
# GATEWAY / CORROBORACAO ESPACIAL (o "hub",
sem telepatia -- so contagem)
#
=====
=====
n_alerting = node_alerts.sum(axis=0) #
quantos nos estao alertando em cada
instante
community_alert = n_alerting >=
CORROBORATION_MIN

#
=====
=====
# COMPARACAO: sensor UNICO (sem rede) vs
REDE com corroboracao
#
=====
=====
single_sensor_alert_time = None
for t in range(T):
    if node_alerts[0, t]:
        single_sensor_alert_time = t
        break

community_alert_time = None
for t in range(T):
    if community_alert[t]:
        community_alert_time = t
        break

```

```

# taxa de falso positivo ANTES do evento
comecar (deveria ser baixa)
pre_event = slice(0, EVENT_START)
single_false_positive_rate = node_alerts[0,
pre_event].mean()
community_false_positive_rate =
community_alert[pre_event].mean()

print("=" * 70)
print("REDE DE SENSORES PARA ALERTA
ANTECIPADO -- RESULTADO")
print("=" * 70)
print(f"Evento real comeca em t=
{EVENT_START} (rampa de {EVENT_RAMP} min)")
print(f"\nSensor UNICO (traje isolado, sem
corroboracao):")
print(f"  Primeiro alerta em: t=
{single_sensor_alert_time}")
print(f"  Taxa de falso positivo (antes do
evento):
{single_false_positive_rate*100:.2f}%")
print(f"\nREDE com corroboracao espacial
(>= {CORROBORATION_MIN} de {N_NODES}
nos):")
print(f"  Primeiro alerta de comunidade em:
t={community_alert_time}")
print(f"  Taxa de falso positivo (antes do
evento):
{community_false_positive_rate*100:.2f}%")

if single_sensor_alert_time is not None and
community_alert_time is not None:
    print(f"\nDiferenca de latencia: rede

```

```

corroborar {community_alert_time -
single_sensor_alert_time} min "
        f"{'depois' if
community_alert_time >
single_sensor_alert_time else 'antes'} do
sensor unico")
print(f"Reducao de falso positivo:
{single_false_positive_rate*100:.2f}% ->
{community_false_positive_rate*100:.2f}%")

# banda usada: 1 bit por no por passo, so
na rede real (nao dado bruto)
bits_transmitted = N_NODES * T * 1 # 1
bit/no/passo (protocolo real de baixa
banda)
print(f"\nDados transmitidos pela rede:
{bits_transmitted} bits totais "
        f"({bits_transmitted/8/1024:.2f} KB)
para {N_NODES} nos por {T} min -- "
        f"compativel com LoRaWAN (~50
bytes/pacote, poucos pacotes/hora)")

#
=====
=====
# PLOTS
#
=====
=====
fig, axes = plt.subplots(4, 1, figsize=(13,
13), sharex=True)

ax = axes[0]
for n in range(min(6, N_NODES)):

```

```

        ax.plot(nodes_temp[n], alpha=0.6,
linewidth=1)
ax.axvline(EVENT_START, color='red',
linestyle='--', alpha=0.5, label='inicio do
evento real')
ax.set_ylabel('Temperatura (°C)')
ax.set_title('Sinal ambiental bruto (6 de
20 nos, nunca sai do dispositivo)')
ax.legend()
ax.grid(True, alpha=0.3)

ax = axes[1]
for n in range(min(6, N_NODES)):
    ax.plot(nodes_hrv[n], alpha=0.6,
linewidth=1)
ax.axvline(EVENT_START, color='red',
linestyle='--', alpha=0.5)
ax.set_ylabel('HRV (ms)')
ax.set_title('Sinal fisiologico bruto
(RMSSD-like, 6 de 20 nos)')
ax.grid(True, alpha=0.3)

ax = axes[2]
ax.plot(n_alerting, color='darkorange',
linewidth=1.5)
ax.axhline(CORROBORATION_MIN,
color='black', linestyle=':',
label=f'limiar de corroboracao
({CORROBORATION_MIN} nos)')
ax.axvline(EVENT_START, color='red',
linestyle='--', alpha=0.5)
ax.set_ylabel('Nos alertando')
ax.set_title('Contagem de nos em alerta
local (o unico dado que sai de cada no: 1

```

```
bit)')
ax.legend()
ax.grid(True, alpha=0.3)

ax = axes[3]
ax.fill_between(range(T),
community_alert.astype(int),
color='crimson', alpha=0.4, label='alerta
de comunidade')
ax.plot(node_alerts[0].astype(int)*0.5,
color='steelblue', alpha=0.7, label='alerta
do sensor unico (traje isolado)')
ax.axvline(EVENT_START, color='red',
linestyle='--', alpha=0.5)
ax.set_ylabel('Alerta (0/1)')
ax.set_xlabel('Tempo (min)')
ax.set_title('Sensor unico vs. rede com
corroboracao')
ax.legend()
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('/home/claude/disaster_sensor_n
et/disaster_alert_network.png', dpi=150)
print("\nArquivo gerado:
disaster_alert_network.png")
```